

CoderCommands MCP System

Step 02 Record — Postgres Schema

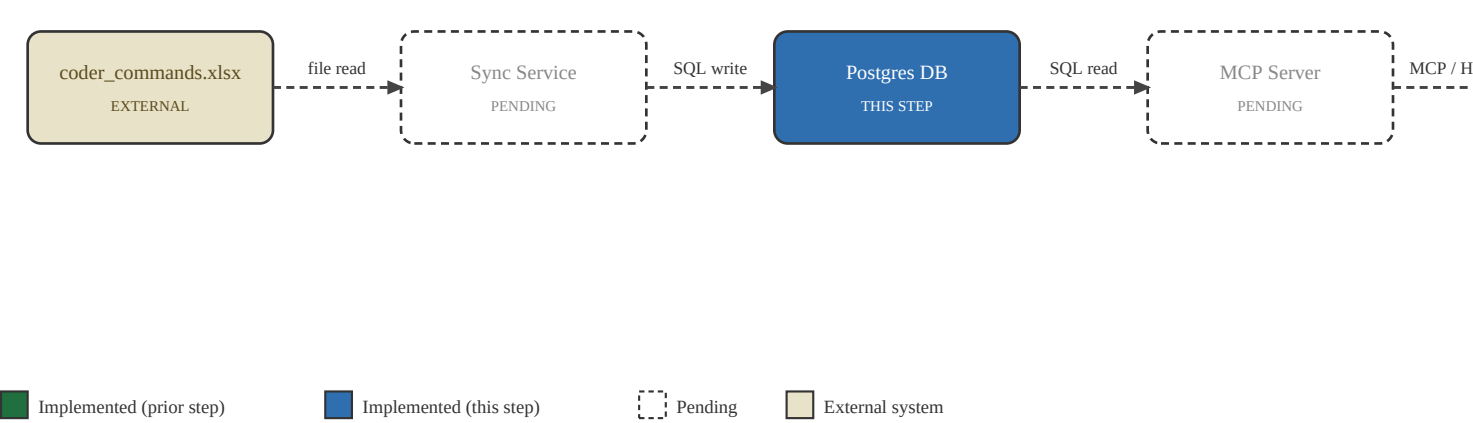
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	02 — Postgres Schema
DATE	2026-08-15
REVISION	1.0
STATUS	Complete

1. Purpose & Scope

Define the single canonical data store the whole system reads and writes: the target for the Sync Service (Steps 3–4) and the source of truth for the MCP Server (Step 5). This is the 'process database' layer of the system — everything upstream (the xlsx) and downstream (search tools) is defined relative to this schema, so it is fixed before any service code is written against it.

2. System Context Diagram

Signal-flow view of the four functional units (Sync Service, Postgres, MCP Server, FastAPI Gateway) plus the two external endpoints (source workbook, Claude Code client). Fill/border indicates build state as of this step.



3. File / Component Register

Role of every file or component introduced or modified in this step, with its upstream and downstream interfaces (I/O-list style).

Ref	File / Component	Role	Interface In	Interface Out
R1	db/schema.sql	DDL applied automatically on first container boot via Postgres's docker-entrypoint-initdb.d convention. Defines the commands table (one row per command-reference entry) and the sync_runs table (one row per sync pass, for audit/troubleshooting).	Sync Service (write)	MCP Server (read)

4. Work Performed

- commands table: topic, section, command, description, extra, row_order, plus a STORED generated search_vector (tsvector) column combining command/description/extra/section.
- GIN index on search_vector for the MCP server's search_commands tool; btree index on topic for the per-topic browse_<topic> tools and for the Sync Service's per-topic delete-and-replace operation.
- sync_runs table: an audit trail of every sync pass (start/finish time, source file mtime, topics/rows synced, status, error) — the equivalent of an event log for the data pipeline, so a failed or stale sync is diagnosable without reading container logs.
- Validated the DDL against a disposable postgres:17 container (matching the version already in use elsewhere on this host) before wiring it into the compose stack in Step 7.

5. Issues Encountered & Resolution

Issue	Root Cause	Resolution
Bind-mounting schema.sql into a disposable test container silently produced an empty /docker-entrypoint-initdb.d/ inside the container, so Postgres skipped initialization entirely on first attempt.	Git Bash (MSYS) rewrites POSIX-looking path arguments to Windows paths before they reach the docker CLI. The container-side path in the -v mount (/docker-entrypoint-initdb.d/...) was being mangled the same way, so the mount target inside the Linux container never matched.	Re-ran the same docker run with MSYS_NO_PATHCONV=1 set, which disables MSYS path rewriting for that command. The file appeared correctly inside the container on the next attempt. This applies to any docker run/exec invocation with POSIX-style paths from this shell -- flagged for the Step 7 compose validation and Step 8 verification runs.
Considered making the tsvector's search configuration ('english') a per-row column to support future non-English notes.	PostgreSQL requires GENERATED ALWAYS AS columns to be built from IMMUTABLE expressions only; to_tsvector() with a non-constant regconfig argument is not immutable and is rejected at CREATE TABLE time.	Hardcoded 'english' as a literal in the generated expression for v1. All current sheet content is English, and the constraint doesn't block adding a fixed-language column override later if that changes.

6. Verification Record

Check	Method	Result
schema.sql applies without error on postgres:17	Disposable container, docker-entrypoint-initdb.d	PASS
commands and sync_runs tables + expected columns exist	\dt / \d commands / \d sync_runs via psql	PASS
GIN + btree indexes created as defined	\d commands index listing	PASS
Full-text search returns the correct row for a natural-language phrase	Inserted 2 sample SQL rows, queried plainto_tsquery('english','insert new row table') → matched 'INSERT INTO tableName...' and correctly excluded the SELECT row	PASS

7. Next Step / Open Dependencies

Step 3 builds the ingest/normalization module that transforms the 14 inconsistent xlsx sheets into rows matching this exact schema.